Fifty-seven Commits

What Claude Code has contributed to Informath, July–October 2026

Aarne Ranta

October 2026

This talk

- Informath: over 900 commits since 2024, almost all by hand.
- **57** of them are co-authored by Claude (Claude Code, Anthropic's coding agent, working in a terminal in my checkout).
- For each task: **why** it was needed, the **prompt and interaction**, and the **outcome** — including what is still wrong.
- Sources: `git log`, the diffs, Claude Code session transcripts (September, October), session logs that Claude wrote itself (July, and the Rijke project in September).

These slides were themselves drafted by Claude from those sources (`35d81f9`, 27 September), and extended in October with Tasks 12–23 (`ab75fd7`, `9809d76`, `ffb638d`, `8cbe9a3`, `09a82f6` and this version).

Outline

Informath in five minutes

Part I: New languages

Part II: Godement's *Algebra*

Part III: Tests, a second book, speed, and checks

Part IV: Passing it on

Part V: Mathlib, and agreement

Observations and the future

Informath in five minutes

The problem

Mathematics exists in two registers.

Formal

- Agda, Rocq, Lean, Dedukti
- machine-checkable
- unreadable to most humans

Informal

- English, French, German, Swedish, ...
- readable
- ambiguous, unverifiable

Informath translates between them in all directions: **informalization** (formal $\rightarrow$ informal), **autoformalization** (informal $\rightarrow$ formal), and informal $\rightarrow$ informal via the formal.

One statement, many languages

```
Dedukti: prop110 : (a : Elem Int) -> (c : Elem Int) ->
  Proof (and (odd a) (odd c)) ->
  Proof (forall Int (b => even (plus (times a b) (times b c)))).
```

- **En**: Let a and c be integers. Assume that a and c are odd. Then $ab + bc$ is even for all integers b.
- **Fr**: Soient a et c des entiers. Supposons que a et c sont impairs. Alors $ab + bc$ est pair pour tous les entiers b.
- **Sv**: Låt a och c vara heltal. Anta att a och c är udda. Då är $ab + bc$ jämnt för alla heltal b.

All generated from the Dedukti source, and Agda, Rocq and Lean as well.

Architecture

Dedukti — the semantic core

↕

MathCore — abstract syntax close to the logic

↕ (semantics / NLG in Haskell)

Informath — variation, synonyms, symbolic vs. verbal

↕ (GF concrete syntaxes, using the RGL)

Eng **Fre** **Ger** **Swe** (+ Latex, + experimental Fin Cze Pol)

- GF: Grammatical Framework; RGL: its Resource Grammar Library (40+ languages).
- The grammar is split into a language-independent *functor* and small per-language modules of words and function words.

Symbol tables: from constants to words

Each Dedukti constant gets a wording and/or a notation, given *by example*:

```
isGroup     : "#1 is a group"
isSubgroup  : "#1 is a subgroup of #2"
inv         : "the inverse of #2" | $#2^{-1}$
cosetRel    : "#3 is congruent to #4 modulo #2"
```

- The example is *parsed* with the grammar into a lexical tree (`NounPrepNoun2 subgroup_Noun of_Prep`), which then works in any statement, in any language.
- `$...$` forms become LaTeX macros.
- Only shapes the grammar can parse are allowed — a recurring theme today.

How Claude worked

- Claude Code runs in a terminal, reads and edits the files, runs `gf`, `stack`, `RunInformath`, `pdflatex`.
- I give short prompts in English; Claude plans, works, reports.
- Commits only when asked; I push myself.
- It can spawn *subagents* for parallel subtasks (e.g. batches of words).
- Remembers preferences across sessions in a small memory file, e.g. “prefer symbolic expressions”.

The first 23 commits as 11 tasks

	Task	Date	Commits
1	Finnish words and function words	10 July	2
2	A Czech grammar	10, 16 July	4
3	A Polish grammar	16 July	2
4	Completing the French Wikidata lexicon	22 July	1
5	Words for the Godement formalization	24–25 Sept	5
6	Clash-free macro names	24 Sept	1
7	Proof texts	25 Sept	4
8	Compressing hypotheses in proof texts	26 Sept	1
9	Formulas in standard logical notation	26 Sept	1
10	GodementSyntax: the language of definitions and theorems	26 Sept	1
11	GodementExamples: richer symbol tables	26 Sept	1

Part I: new languages. Part II: formalizing Godement's *Algebra*.

Since 27 September: 22 more commits as 10 tasks

	Task	Date	Commits
12	Godement-style readings	27 Sept	1
13	Godement §§ 25–36: words and Greek variables	28 Sept	1
14	Regression tests, and what they found at once	29 Sept	4
15	A second book: Rijke's homotopy type theory	29–30 Sept	2
16	Speed of generation	14–15 Oct	4
17	Comments in symbol tables	14 Oct	1
18	Type checking of symbol tables	16 Oct	2
19	A template for autoformalization experiments	16 Oct	1
20	Open issues for the next machine	16 Oct	1
21	The template's first experiment	19 Oct	5

Plus five commits for these slides. Part III: tests, a second book, speed, and checks. Part IV: passing it on.

Since 21 October: 7 more commits as 2 tasks

	Task	Date	Commits
22	Informath for the informalization of Mathlib	22 Oct	4
23	Agreement of "such that", and a time explosion	22 Oct	2

Plus one commit for these slides. Part V: Mathlib, and agreement. Task 22 is told in full in its own deck, `mathlib-informath/doc/informalizing-mathlib-slides.pdf`.

Part I: New languages

Task 1: Finnish — why

- A Finnish grammar existed as a *compilable template*, made by copying Swedish and English.
- It said `linking ... OK` — and it said `then_Adv = mkAdv "så"`.
- `VerbalConstantsFin`: an English copy with two Finnish words.
- `WikidataWordsFin` (2000+ terms) was a byte-identical copy of English and did not compile, so it was commented out:

  a grammar that "worked" by excluding 2000 of its 2100 words.

- The question: can an LLM complete a GF grammar for a language it knows well, given the RGL?

Task 1: Finnish — prompt and interaction

Goal given (as recorded in Claude's session log):

> *"Complete the Finnish concrete grammars by replacing placeholder (mostly Swedish / English-copy) content with proper Finnish mathematical vocabulary."*

- The prompts themselves were not saved; this was the first experiment.
- Three layers: ∼23 function words, ∼120 core math words, 2051 Wikidata terms.
- The Wikidata terms went to **7 parallel subagents**, each returning an English–Finnish table; a Python script generated GF.
- Claude found why the lexicon did not compile, and dropped the dependency on the Finnish dictionary: Finnish *smart paradigms* suffice.
- My interventions: asking about prepositions; not accepting `linking ... OK` as proof of correctness.

Task 1: Finnish — outcome

`9ac0431`, `ab82b63` (10 July)

- 2051 entries: 1216 nouns, 749 adjectives, 86 verbs; all enabled; links.
- *lyhde* (sheaf), *ydin* (kernel), *jaoton* (prime); *olkoon x ryhmä*; `mkNoun "rengas"` gives *kaikille renkaille R* with consonant gradation for free.

Remaining problems

- Five prepositions are loose words (`mkPrep "yli"`) where Finnish uses cases: "They compile. They read acceptably. They are not right."
- Invented terms: quiver → *nuolikko*; capitalized *Abelin* next to *abelin*; 33 orphans, 15 duplicates inherited from English.
- No native review yet; moved to `grammars/next/`.

Task 2: Czech — why

- Czech: a new Informath language, and a test of a *weak* RGL.
- The template was Swedish down to the morphology: `mkN "typ" "typer"`.
- The Czech RGL was a stub: `SentenceCze` had 4 rules, no `mkV`, no `V3`; missing rules crashed only late: `NOT YET IMPLEMENTED: ComplVS`.
- Informath's functor calls the RGL by qualified names, so the gaps could not be worked around inside Informath.

Task 2: Czech — prompt and interaction

"Make `InformathCze.gf` compile, link, and produce Czech."

- A loop: crash, write the missing RGL rule, rebuild the library — about 20 times. "Tirelessly."
- Half of the session was spent in the RGL (commits there too).
- 2003 terms by **8 subagents**, with French and German as hints.
- Asked to audit itself, Claude re-implemented the RGL's paradigm selector in Python and compared: 225 of 1177 nouns (19%) fell back to a wrong declension — two RGL bugs, affecting every Czech grammar.
- A human question: "is that actually an imperative?"

Task 2: Czech — outcome

`e81159b`, `f756218` (10 July), `114fbd4`, `ab7e423` (16 July)

- InformathCze compiles and links; its own verb conjugator for *-ovat, -at, -it/-et/-ět*.
- 2003 terms, nouns with nominative, genitive and gender; wrong fallbacks from 225 to 1 after 7 new paradigms.
- 789 words in Godement's register; three-place verbs after an RGL fix.

Remaining problems

- *existuje matice M, takový, že* — should be *taková*: "such that" cannot agree in gender. — In part: Task 23.
- Multi-word nouns decline only the head; "imperatives" are present tense.
- UserExtensionsCze still disabled; ∼1500 terms unreviewed.

Task 3: Polish — why

- The second Slavic language, now with Czech as the model.
- The Polish RGL was in good shape (only 6 missing rules) — the problem turned out to be *nouns*.
- A test of whether the lessons of Czech generalize.

Task 3: Polish — prompt and interaction

> *"Create the Polish concrete syntax modules, following the Cze structure, so that InformathPol.gf compiles."*

and then, on request, Mizar words and user extensions.

- Three dead ends with nouns:
  - `mkN` guessed *zbióra, elementowie, przypadeka*;
  - the two-argument `mkN` dropped the genitive; fixing it gave *liczbaa* — reverted and documented;
  - the 1057 SGJP tables bake in an example word: *samolot* turns *wektor* into *wektot*.
- So Claude wrote its own declensions, taking the stem from the genitive.
- 12 subagents; they *refused* to fake genitives for adjectival nouns, which led to a new paradigm.
- Random generation found *dla wszystkich płytki*; a test of `meet_Verb2` found a word-order bug in the RGL.

Task 3: Polish — outcome

`6ee105f`, `c735985` (16 July)

- InformathPol links: 2003 terms, 0 rejects; *niech x będzie grupą*.
- Mizar words reuse Wikidata terms where the sense matches: *pole relacji*, not *ciało*.
- The session log ends with six lessons "for the next language".

Remaining problems

- *istnieje macierz M, taki, że* — the same agreement problem as Czech.
- *koł* for *kół*; weak terms (*pushforwardowy*).
- Needs a hand-compiled `MarkupPol`: `make next_grammar` fails on a fresh machine.

Task 4: French Wikidata — why and interaction

Why

- French was an official language, but `WikidataWordsFre` was hand-curated: 537 of 2003 terms, no verbs.

Prompt and interaction

> *"Complete `WikidataWordsFre.gf` so that every abstract WikidataWords fun has a French linearization."*

- A diff: 1481 missing (1008 nouns, 387 adjectives, 86 verbs).
- **15 parallel agents**, ∼100 terms each; 15 name collisions dropped.
- A second pass on multi-word plurals: *trous spectraux*, *chemins de fer*.
- One smooth session.

Task 4: French Wikidata — outcome

`9699638` (22 July): 2003/2003 terms, compiles.

```
lin close_Verb = mkVerb (mkV "fermer") ; ---Claude
```

- Every new line is marked `---Claude` (1466 of them): `grep -v` gives a lexicon without them until they are checked.

Remaining problems

- Uncertain senses: *stack* → *champ*, *net* → *filet*, *quiver* → *carquois* (a quiver for arrows).
- ∼21 invariant multi-word adjectives; source typos carried over (`minmum_Noun`).
- All unreviewed. Next candidates: German (∼1650 missing), Swedish (∼1850).

Part II: Godement's *Algebra*

The Godement project

- Roger Godement, *Algebra* (1968): a classic textbook, Part I (sets) and Part II (groups, rings, modules, linear algebra).
- Since 18 September, a companion repository `godement-index`:
  - the whole book as a UD treebank: 10 413 sentences;
  - a Dedukti formalization of Part I and §§ 6–24, with Claude, each constant linked to Godement's own sentence;
  - symbol tables, English readings, formulas, and an HTML index.
- The Informath commits of Part II were *needed by* this work: missing words, bugs found, new outputs, new grammar.

Task 5: Words for Godement — why and interaction

Why

- Every symbol-table entry must parse. Godement's wordings needed words the lexicon lacked: "R is *undecidable*", "X is *equipotent* to Y", "a *two-sided* ideal", "a *Cramer* system".

Prompt and interaction

> *"let us continue from PARTI-FORMALIZATION.md to complete the Godement book till the end of chapter 5"*

- Claude added a word whenever a table entry failed to parse, and committed it with the formalization (*"make a commit if ready for it; I will push"*).
- It deliberately did *not* add `countable_Adj`: already in WikidataWords, and a second declaration breaks the build.

Task 5: Words for Godement — outcome

`583e0e5`, `a4427ce` (24 Sept), `bc4d431`, `00c01cb`, `6509853` (25 Sept)

```
fun 'two-sided_Adj' : Adj ;
lin 'two-sided_Adj' = mkAdj "two-sided" ;
```

- 9 words: *undecidable, contradictory, equipotent, generate, two-sided, finite-dimensional, alternating, increasing, Cramer*.
- Symbol tables for Part I and §§ 6–24 build.

Remaining problems

- Words were the easy part: the *shapes* of entries were limited — no three-place nouns, no "into", no negation (see Task 11).

Task 6: Clash-free macro names — why

- A LaTeX form `$...$` in a symbol table becomes a macro.
- Its name was the letters of the constant + `MACRo`: digits and underscores were dropped.
- `card0`, `card1`, `card2` all became `\cardMACRo`; the last definition silently won.
- Nothing warned. The document compiled and said

$$2 \cdot x = 2 \quad \text{for} \quad 0 \cdot x = 0, \qquad x^2 = x \quad \text{for} \quad x^1 = x.$$

Task 6: Clash-free macro names — prompt and interaction

- Found while reading Part I back. First: *"what are you doing?"* and *"I see that it stalls. Can you just jump over that?"*
- Claude proposed two fixes: spell digits as words (`\cardZeroMACRo`), or rename the constants.

  "I know these fixes. They would not solve everything, because there can be names that already contain Zero etc. It would be better to generate all macro names with an algorithm that guarantees uniqueness. For instance, suffixes MACRoA, MACRoB, ... MACRoAA, MACRoAB, ..."

- Implemented as a bijective base-26 index over the whole table; *"this sounds good"*.
- Claude corrected its own diagnosis twice after measuring (the stall was one oversized theorem, not the flags).

Task 6: Clash-free macro names — outcome

`f445edd` (24 Sept)

	before	after
macros	`\cardMACRo` (3 constants)	`\cardMACRoY, ...Z, ...AA`
`mulCard_zero`	$2 \cdot x = 2$	$0 \cdot x = 0$
`powCard_one`	$x^2 = x$	$x^1 = x$
`projections`	$\mathrm{pr}_2(x, y) = x$	$\mathrm{pr}_1(x, y) = x$

Remaining problems

- Names depend on position: adding a LaTeX form renames later macros, so derived files must be rebuilt together.
- Informath has no regression test suite; checking is by comparing outputs. — It has one now: Task 14.

Task 7: Proof texts — why

- Godement's theorems now had real Dedukti *proofs* (§ 6 onwards).
- Informath could print a proof as numbered lines, but:
  - every argument of a rule became a line, mostly "there is a proposition": § 6 Theorem 1 gave 46 lines, 14 of them steps;
  - the output was a LaTeX 2.09 demo (`\documentstyle`) that did not compile;
  - lines were raw token lists: `$ y $ is equal to $ x $ .`
- Goal: proofs that a mathematician can read, as PDF.

Task 7: Proof texts — prompt and interaction

- No transcript survives for this day; the story is in four commits made within half an hour:
  1. show the steps, not the arguments (a heuristic filter);
  2. replace the heuristic by *profiles*: `#SHOW forallE [3,4]` in the symbol table, or read the premisses off the rule's type;
  3. a document that can be read and compiled;
  4. unlex the lines.
- Each was committed together with the regenerated proofs in `godement-index`.
- A slip: commit 2 swept in unrelated files (slides, logs, `*.gf~` backups).

Task 7: Proof texts — outcome

`0623e2b`, `dabdd50`, `d183036`, `2ef5f55` (25 Sept)

```
5. Assume that x is equal to y.
6. Assume that y is equal to z.
7. x is equal to y and y is equal to z.        andI 5, 6
8. x is equal to z.                            ifE 4, 7
```

- PDFs of the proofs for §§ 6–10, 12–17, 22 (27 pages for § 6).

Remaining problems

- Each proof is headed by its raw Dedukti type, not an English theorem.
- § 6 Theorem 1 prints two lines: the declaration, then its *conclusion* as an assumption; the steps are lost.
- Proof labels leak into formulas ("the value of f at `hy`").

Task 8: Compressing hypotheses — why and interaction

Why

- Every variable had its own line: "Let x be ... Let y be ... Let z be ..." — long and unlike mathematical prose.

Prompt and interaction

> *"I want to improve src/ProofText.hs by compressing hypotheses. If there are consecutive proof lines introducing a variable with the same type, they should be compressed to one line using the abstract syntax function GVarsHypo on the list of those variables. The lines should of course be renumbered to reflect this."*

- Claude also found that the Dedukti table (`-proof-terms`) would no longer match, and that the flag had never been accepted at all.
- *"Do compress the Dedukti table. But where in the code are the wrong variable names created?"* — diagnosed (beta reduction under the rule's binder), fix proposed, not made.

Task 8: Compressing hypotheses — outcome

`c391de6` (26 Sept)

Before (8 lines)

```
1. Let x be a mathematical object.
2. Let y be a mathematical object.
3. Let z be a mathematical object.
4. If x is equal to y and ...
```

After (6 lines)

```
1. Let x, y and z be mathematical
   objects.
2. If x = y and y = z, then x = z.
3. Assume that x = y.
```

- The symbolic $x = y$ came from another change; asked about it: *"Yes, I did want to increase the usage of symbolic expressions instead of verbal ones."* — saved as a lasting preference.

Remaining problems

- Wrong bound-variable names ("for all elements x and m of $\mathbb{N}$") are still there.

Task 9: Standard logical notation — why and interaction

Why

- Readers want each statement also as a compact formula ($\forall$, $\wedge$, $\to$, $\vdash$), next to the English.
- Informath had symbolic notations for constants, but not for the logic.

Prompt and interaction

> *"I would like to have grammars/MathCoreLatex.gf and InformathLatex that generates formulas with standard logical notation and uses the symbolic notations defined in SymbolicConstants-Latex and in .dkgf files. In RunInformath, the flag -to-symbolic-latex should translate Dedukti to this notation. [. . .] Use the precedences in the same way as in TermsLatex."*

- Claude chose conventions itself and offered to change them: sequents $\Gamma \vdash A$, $:\Leftrightarrow$ for definitions, no proofs shown.
- *"Also update godement-index.html with these formulas."*

Task 9: Standard logical notation — outcome

`f87c660` (26 Sept): three new GF modules, a flag, a Makefile target.

- English: "Let f be a function from mathematical objects to mathematical objects. Let u and v be mathematical objects. Assume that $u = v$. Then $f(u) = f(v)$."
- Formula: $f \in \mathrm{Obj} \to \mathrm{Obj},\ u, v \in \mathrm{Obj},\ u = v \ \vdash\ f(u) = f(v)$
- All 1138 declarations in 5 seconds; a "Formula" fold in the index.

Remaining problems

- Constants without notation print as $\mathrm{isGroup}(g)$.
- Conjunctions nest with explicit brackets: $A \wedge (B \wedge (C \wedge \dots))$.
- Claude's conventions not yet confirmed; long formulas only scroll.

Task 10: GodementSyntax — why

- Informath's grammar covers the statements that come *out* of Dedukti.
- Godement *writes* differently. Counted in the 599 formalized sentences (UD trees, regex + dependency patterns):

Construction	**sentences**	**in the grammar?**
"x is said to be A if ..."	53	no
"x is called the N of y"	65	no
"for x to be A it is necessary and sufficient that"	24	no
"the following conditions are equivalent: a) ... b) ..."	33	no
"there exists a unique x such that"	22	no
"subgroup of G which contains B" (relative clause)	105	no
"if P then Q", "for all x", "there exists"	180+	yes

Task 10: GodementSyntax — prompt and interaction

"Let us investigate the linguistic structures used in definitions and theorem statements in Godement. [...] Can you make a list of the structures of statements in Godement by using the Informath categories Prop, Kind, Exp, Adj, etc? [...] Do not try to run GF in parsing mode yet, because it can be slow."

"Excellent! Can you suggest GF abstract syntax functions, linearizations using RGL, and mappings to MathCore"

"You can add GodementSyntax to UserExtensions and a test case [...] and a Makefile entry for godement_syntax that produces a pdf and opens it."

- Parsing real sentences revealed what linearizing did not: `$...$` is indexed before parsing, a hidden argument crashes the parser, a comma too many blocks a parse.

Task 10: GodementSyntax — outcome

`865eae0` (26 Sept): about 50 functions; semantics in Haskell; a flag `-translate-core`; `make godement_syntax` → PDF.

- *For an element x of G to be reflexible, it is necessary and sufficient that x has a left reflection.*
  $\Rightarrow$ for all elements x of G, (x is reflexible, if and only if there is a left reflection of x).
- *There exists a unique element x of G such that x is invertible.*
  $\Rightarrow$ there exists $x \ldots$ and for all $y \ldots$, if y is invertible, then $y = x$.
- 38 of 38 test sentences parse; no regressions on the old tests.

Remaining problems

- Ambiguity: several readings per sentence; ugly fresh variables _g0.
- English only; "x is a K" has no Dedukti translation when K is a type.
- Not yet run on Godement's actual 10 413 sentences.

Task 11: GodementExamples — why

- Symbol tables speak Godement only as far as `Examples.gf` lets them.
- The formalization notes (written with Claude) listed the gaps:
  - no three-place noun: "a homomorphism of G into H" became "f is a homomorphism";
  - no "into", "onto", "generated by";
  - no negation: "x is not an element of y";
  - "zero" and "one" but no "two".
- 216 table entries showed fewer arguments than their constants had.

Task 11: GodementExamples — prompt and interaction

> *"We noticed that some syntactic forms that should be usable in .dkgf entries are missing from the grammar. [...] I would like to start with a new module GodementExamples [...] You can try to rebuild the dkgf files by using these new functions where necessary. If this means that you need some new categories, add them and their application functions also to GodementExamples."*

- New categories need more than grammar: the Dedukti $\leftrightarrow$ MathCore conversions in Haskell had to learn them.
- Verified by reading chapters back with old and new tables and diffing.
- *"go on"*: a second round of rewordings, and the whole index rebuilt.

Task 11: GodementExamples — outcome

`35184d0` (26 Sept): 3 new categories, 5 example forms, 12 prepositions.

before	after
`"#1 is a homomorphism"`	`"#1 is a homomorphism of #2 into #3"`
(not worded)	`"the subgroup of #1 generated by #2"`
`"the class of #3 under #2"`	`"the class of #3 modulo #2 in #1"`
`$#1 \notin #2$` only	`"#1 is not an element of #2"`

Remaining problems

- "the set of all subsets of X with n elements" still does not parse.
- Negated definitions read as tautologies: "$x \notin y$ if $x \notin y$".
- The next morning: English readings of §§ 10–24 had never been possible; 21 declarations are too big to word in 30 seconds. — Task 16.

Task 12: Godement-style readings — why and interaction

Why

- Informath read every statement in one way: "Let g be a mathematical object. Assume that g is a group."
- Godement writes "Let G be a group", "a group G is said to be commutative if", "for all elements s of G".
- To compare the formalization with the book, the readings should be able to sound like the book.

Prompt and interaction

- No transcript survives: weekend work, one commit.
- GodementSyntax (Task 10) *parses* Godement's constructions into MathCore; the new GodementNLG is its inverse and *generates* them.

Task 12: Godement-style readings — outcome

`31343d5` (27 Sept): `src/GodementNLG.hs`, flags `-godement` and `-trees`.

- Variants: typed variables, "is said to be", "is called", "for all elements s of G", "necessary and sufficient".
- `godement-index` compares the 50 best readings (`-trees`) with Godement's own sentence and shows the closest:

Godement	Informath	sim.
The field of complex numbers is algebraically closed.	The field of all complex numbers is algebraically closed.	0.84
The rank of the homomorphism f is defined to be the dimension of the subspace $\mathrm{Im}(f)$ of M.	Let f, I, m and k be mathematical objects. Then the rank of f is defined to be the dimension of the image of f.	0.65

Remaining problems

- Mean similarity 0.25 over 1534 constants; many variables are still "mathematical objects"; generation was slow (Task 16).

Task 13: Godement §§ 25–36

Why

- The last part of the formalization: polynomials, eigenvalues, hermitian forms; written so that the Godement-style readings begin as the book does.

Prompt and interaction

"I have not continued from chapter 25 but done other things. Therefore, I want you to continue from chapter 25 of Godement now. Please read what was added to the grammar during the weekend and how it is used to make the informalizations more like the original text."

- Later: *"did you try the -force-symbolic fallback?"* — for declarations that have no wording.

Outcome

- `7a0a687` (28 Sept): *algebraically closed*, *self-adjoint*, *positive definite*; *associated with*, *with coefficients in*, *of degree*; Greek variables print as σ, a trailing digit as a subscript.
- "Let K be a field. Let σ be an involution of K. . . ."

Part III: Tests, a second book, speed, and checks

Task 14: Regression tests — why and interaction

Why

- Task 6: "Informath has no regression test suite; checking is by comparing outputs."
- The first version of these slides (27 Sept) proposed it first among the next steps.

Prompt and interaction

- No transcript on this machine. Four commits within 15 minutes, on the morning of 29 Sept:

09:08	`fa66483`	the test suite
09:12	`e2b0e8e`	a bug the tests found
09:18	`91a4106`	all readings of formulas
09:23	`63f8b09`	`-typecheck`, to filter the extra readings

Task 14: Regression tests — outcome

- `regression/run.py`: the Makefile demos as 49 cases in four tiers, in about 15 seconds. Checks: runs in time, LaTeX compiles, no new failure lines, no more undefined constants, not twice as slow; the `check` tier runs `dk`, `agda`, `coqc`, `lean`.
- Found at once: Dedukti parsed back from English said `n : Nat` for `n : Elem Nat` — a coercion toggled instead of added, by semantics applied twice. Well-typed round trips: 7 → 12 of 15; Chartrand's examples: 0 → 12 of 19.
- `$a \times b$` had only ever been read as a cartesian product: now all readings of a formula are kept, and `-typecheck` keeps those that `dk check` accepts.
- Since then every commit says "48 PASS, 1 NEW, 2 XFAIL" or why not.

Remaining problems

- No CI; two known failures (`fermat`, `bind`); German has no expected output yet.

Task 15: A second book — why and interaction

Why

- Egbert Rijke, *Introduction to Homotopy Type Theory*, in a third repository, `hott-llm`. Type theory stresses Informath differently: explicit type arguments, rewrite rules, constants with only a LaTeX form.

Prompt and interaction

> *"Read Rijke's book and formalize the first chapter. Use a similar set-up as in the session with ../godement-index. That is: formalize in Dedukti, produce a symbol table for Informath, extend the grammar if necessary, and report about issues that need to be fixed in Informath. Generate an index file similar to godement-index.html."*

- A correction: *"Primary is not so important any more: it is just a part of verbal. Symbolic constants are between dollars and verbal constants between quotes."* — Claude's first fix was reverted and redone.
- On request, Claude kept `LOG.md`, a log of the sessions.

Task 15: A second book — outcome

`277eaac` (29 Sept), `1ad457d` (30 Sept)

- `$#4 \circ #5$` made a macro of five parameters, but macros take at most four: $\mathrm{comp}(A, B, C, g, f)$. Now a macro takes the arguments its form uses, and the constant gets a matching profile.
- The same bug in Godement, unnoticed: `\mathrm{matrixProduct}` 50 times.
- Rewrite rules: $\mathrm{refl}_x \cdot q :\equiv q$ instead of $\mathrm{concat}(A, x, x, z, \mathrm{refl}_{A,x}, q) :\equiv q$.
- By 30 Sept: §§ 1–22, 673 items, 414 formalized, `rijke-index.html`.

Remaining problems (from `NOTES.md`)

- Curry–Howard theorems read as definitions; λ-arguments split as binders; "g applied to f applied to x".
- Colons in table comments (Task 17); speed (Task 16).

Task 16: Speed — why

- Task 11: 21 declarations too big to word in 30 seconds.
- `build-informal.py` worked around it: 120 s per chapter, then declaration by declaration, then `-force-symbolic`.
- Chapter 20 alone ran for more than 10 minutes; Cramer's formula for 2×2 systems never finished.

  "can you find out why generation of natural language from Dedukti is so slow? [...] My hypothesis is that the problem is the large number of variations and their ranking. But maybe there is a low-hanging fruit in some basic routing that can be find by profiling the Haskell code of RunInformath. Can you try this out and come back with suggestions?"

Task 16: Speed — interaction

- Timing each declaration: one of chapter 20's 20 declarations took almost all the time; even `-mathcore`, which skips the variants, ran out of time.
- A profiling build: 78% of the time went to rejecting symbolic/verbal mixes *after* generating all 2^n of them, n the number of constants; ranking took 3%. The hypothesis was half right: too many variants — but of annotations, before the ranking.
- *"Did you try the options -synonyms=<int> and -symbolics=<int>?"* — already 1: each constant still has a symbolic and a verbal form.
- *"yes, go ahead with the pruning and then the cap"*; the next day, *"continue with the two suggested tasks"*: one paid off after a detour, the other gave nothing and was reverted.
- A slip: *"It is ready to push I guess"* — and Claude pushed. *"I want to push manually in the future"*: saved as a memory.

Task 16: Speed — ranking without lexing

"LaTeX tokenizing could be avoided, if you don't unlex when ranking."

- Each candidate was linearized, unlexed — and lexed again, just to count its tokens. The words of the linearization are its tokens already.
- Not quite the same count: GF gives `\frac{` or `}^{` as one token, the lexer gave several. Some variants change places:

	old	**new**
... the union of A and the complement of A ...	176	176
... the union of A and $A^{\complement}$ ...	178	174

(penalties: the lower wins)

- *"It is actually good that the symbolic readings win now."*
- And: *"the (un)lexer needs to be fixed at some point"* — it leaves stray spaces: `${A }^{\complement}$`.

Task 16: Speed — outcome

`7e7f427`, `f243e24` (14 Oct), `5416ea5`, `92f49eb` (15 Oct)

	before	**after**
nested ring expression, depth 6	45 s	0.1 s
Cramer's formula, 2×2	stopped after 60 s	2 s
chapter 20, Godement readings	50 s and a time-out	6 s
the whole book, three readings + index	time-outs, one by one	3 min
the whole book, informal readings	67 s	38 s
the whole book, Godement readings	104 s	70 s

- Pruning and counting: identical output on every chapter. The cap (256 mixes per subformula): 107 of 109 declarations as close to Godement, 2 closer.

Remaining problems

- The time now goes to linearization and Dedukti → MathCore; the lexer and unlexer disagree on tokens.

Task 17: Comments in symbol tables

Why

- Rebuilding `godement-index` with the faster Informath gave an empty file of macros: every formula of the index broken.

Prompt and interaction

- A `.dkgf` line was an entry unless it started with `#`, and `partI.dkgf` begins with a comment with a colon. Since Task 15 every entry is type-checked, so the comment crashed the macro command, and the script wrote an empty file without complaint.
- Met on 29 Sept in `hott-llm`, worked around by rewording the comment.
- Of three options offered, chosen: *"Fix Informath"*.

Outcome

- `94266b2` (14 Oct): comments `(; ... ;)` are skipped, also across lines and nested; `build-informal.py` now stops on the error.

Task 18: Type checking of symbol tables — why

- A symbol table can be checked against its Dedukti file: does each form have the arity and the category of its constant?
- In Rijke's § 9 the check gave 100 messages `MISMATCHING TYPES`, such as

  ```
  htpy ("Kind",4) <> NounPrepDep2 homotopy_Noun from_Prep to_Prep (...,2)
  ```

  for `htpy : "homotopy from #3 to #4"`, which is right.
- Too many false alarms, and so no use.

  "I would like to work on the type checking of Informath symbol tables. There is something wrong and probably missing in BuildConstantTable.symbolTableErrors. For example, the command 'RunInformath -base=section09.dk section09.dkgf' in ../hott-llm gives lots of MISMATCHING TYPES. Can you take a look at this issue?"

Task 18: Type checking of symbol tables — interaction

- Two bugs:
  - every form was compared with *all* the arguments of its constant, although `"homotopy from #3 to #4"`, or a macro since Task 15, shows only some;
  - a function argument `R : Elem A -> Prop` counted as two arguments, which is right only for binders. So `Pi : $\prod_{(#2 : #1)} #3$` passed, though `Pi` has two arguments.
- What was missing: `ARGUMENT OUT OF RANGE`, and checking each form once. A check of argument categories was left out: GF's slots are almost all `Exp`.
- The 22 remaining messages over 55 tables were real, and 18 of them one more bug: a macro using no arguments was printed with all of them, `\ringZeroMACRoBB {r}`, typeset as "0r".
- *"yes, go ahead with the macroProfile fix"*, and *"also rebuild proofs and then commit"*.

Task 18: Type checking of symbol tables — outcome

`a3d0298`, `9626f97` (16 Oct)

	before	after
messages, Rijke's § 9	100	0
messages, the 55 tables of hott-llm and godement-index	about 1600	22, then 4
macros printed with arguments they do not take	929 + 85	0

- A regression case `symboltable-types`, which the old checker fails both ways.
- Both projects rebuilt, with their proofs: § 36 got its proof document.

Remaining problems

- In the tables: `Pi` and `Sigma` with `#3`; `continuum_hypothesis`, a proposition worded as a name.
- In the checker: a function type behind a definition (`Fam Nat`) is not seen; argument categories are not checked.

Part IV: Passing it on

Task 19: A template for experiments — why

- Two experiments, `godement-index` and `hott-llm`, each with its own scripts and its own notes of "things learned the hard way".
- The July notes in `../autoformalize` predate most of it: they say to ignore `MISMATCHING TYPES`, which Task 18 made meaningful.
- Someone else, or a fresh session, could not start a third experiment without redoing the work.

  "I would like you to create a CLAUDE.md file that enables people to do autoformalization experiments similar to godement-index and hott-index. The process should be: read a LaTeX file, formalize its mathematical content in Dedukti, maintain correctness with dk check, build Informath symbol table with both verbal and symbolic expressions as close to the LaTeX as possible, maintain its correctness by checking with RunInformath, create a grammar extension if needed, and present the results [...] You can consult the files in ../autoformalize for hints or create skills corresponding to them, whatever is the best."

Task 19: A template for experiments — interaction

- Two questions first: where (*"Template dir in informath"*) and how much (*"CLAUDE.md + skills + scripts"*).
- The scripts generalized from `hott-llm`: items numbered as LaTeX numbers them, from the document's own `\newtheorem` lines.
- Every shape the symbol-table skill claims was tried with `-parse-example`: three of Claude's own claims were wrong ("zero" is a noun, a typed binder does not parse).
- A test of the *instructions*: a fresh agent, given only the template, formalized a new text on relations, with a proof Dedukti checks, and reported five gaps — all fixed, among them advice on relations as variables, as "Let R be a function from elements R of A ..." showed.
- One more false claim of Claude's, caught by checking it against `godement-index` before the commit.

Task 19: A template for experiments — outcome

5182766 (16 Oct): `informath/autoformalize/`

```
CLAUDE.md           deliverables, the loop, conventions, lessons
.claude/skills/     symbol-table, grammar-extension
scripts/            init, check, check-table, extract-items,
                    build-informal, build-proofs, build-index
base/nd.dk          natural deduction for proofs
example/            divisibility.tex, formalized
```

- To start: copy it, add a LaTeX file, run `scripts/init.sh` and `claude`: *"Formalize section 1"*.

Remaining problems

- Informath issues the trial found: a function-typed hypothesis reads with a variable clash, and as $\mathrm{symmetricF}(A, R, R)$; proof texts can drop a hypothesis; typed binders have no example form.
- Not yet used on a real text by anyone else. — Task 21.

Task 20: Open issues for the next machine

Why

- Claude's memory of open items lives on the machine of the session; the work was to go on from another computer.

Prompt and interaction

"I'm happy with this week's work. Let us continue later. Can I do it from another computer?"

- Claude said what travels (the repositories, CLAUDE.md of the template) and what does not (its notes), and offered to write the open items into the repository; *"OK, update todo"*.

Outcome

- `4653b9d` (16 Oct): `todo/open-issues.md` — the lexer and unlexer, what is left of the speed work, the errors in the symbol tables, the issues of the template's trial, each with how to see it.

Task 21: The template's first experiment — why and interaction

Why

- The template (Task 19) had been tried only on texts written for the test.
- A real text: Reyna and Sandoval's chapter on *classes of subsets* (rings, σ-rings and σ-algebras of subsets), the experiment `mytext`.

Prompt and interaction

- No transcript on this machine: the work was done on the other computer, the one the open issues of Task 20 were written for.
- Five commits on 19 Oct, between 12:29 and 19:09. The experiment needed words, and showing its proofs in its index exposed flaws of the proof texts, each fixed in Informath with a minimal example in the commit message.
- The same day, the indexes of `godement-index` and `hott-llm` were split into one page per unit, with the proofs in English, and the template took that up.

Task 21: The template's first experiment — the proof texts

`d71a179`, `1160444`, `8be976d`

- A lemma of the module itself, used in a later proof, had no type and came out as "Assume that Q"; now it is a rule: "Q" by `lem`.
- A premiss that is a function into proofs, `d : (A : Elem C) -> (B : Elem C) -> Proof P -> Proof Q -> Proof R`, read "Let d be a function from elements A of C ... to elements of Proof applied to R"; now "Assume that for all A and B in C, if P and Q, then R" — Rijke's issue 5.
- A hypothesis applied to premisses yields a conclusion, not an assumption.
- `forallI` renamed the statement's variables ("for all subsets x and B" for A and B); now kept.
- The same conclusion was drawn again for each use (`andEr 2` four times in a row); now once, and cited.

- New regression case `proof-text-lemmas`; the other cases unchanged.

Task 21: The template's first experiment — outcome

`9ff7064`, `c45f7b7` (19 Oct), and the three of the proof texts

- `MytextWords`: the first words module of an experiment made from the template — σ-algebra, σ-ring, *closed under countable unions* . . . , so that "#1 is a ring of subsets of #2" parses.
- The template's index is now a table of contents and one page per unit; a proved declaration shows its proof in English, line by line, with its rules and premisses; no PDFs, so no XeLaTeX.
- With a CoNLL-U parse of the source, each item shows the dependency trees of its sentences, as in `hott-llm`.
- `"informath":` `<checkout>` in `experiment.json`: the scripts use that checkout and its `stack build`, whatever the environment says.

Remaining problems

- `IfI` and `ForallI` now read as tautologies: "Assume that if A, then B. Then if A, then B."

Part V: Mathlib, and agreement

Task 22: Informath for the informalization of Mathlib

Why

- A new project, `mathlib-informath` (21–22 Oct): a Lean exporter of Mathlib's statements and term proofs to Informath's Dedukti, read in English, French and as formulas; 705 statements, 240 proofs so far.
- Most of its commits are in its own repository; four needed Informath.

Outcome

- `4f31f55`: `-proof-text` crashed on a λ typed by a proof of an implication, a negation or a $\forall$; new regression case.
- `ceb9b85`, `7773194`: `Mathlib.gf`, words from Mathlib's docstrings: *smallest, bounded above, partially ordered, set of divisors* ...
- `778e998`: French. `MathlibFre` and partial French modules; proof texts were *empty* in French (`ProofUnitsFre` lacked two functions); `-languages=Eng,Fre` for `-json` and `-proof-text`, so that the grammar, 2.8 s to load, is loaded once (my request); `-no-back-to-dedukti`.

Task 23: Agreement of "such that" — why and interaction

Why

- `such_that_Subj` is a conjunction, so "such" cannot agree: Czech "existuje matice M, takový, že", should be *taková* (slide of Task 2).
- I had added `such_A` and `suchThatCN`, an AP agreeing with the CN, and used it in `SuchThatHypo` (`bf2e121`).

Prompt and interaction

"a similar change should be done in all grammars/.gf. The modules grammars/next/Utilities* should also be updated so that they compile and work properly."*

- A test in seven languages showed two problems, and Claude asked:
  - German and Finnish put the AP, with its clause, *before* the noun: "eine dass *n* prim ist derart natürliche Zahl" — `suchThatCN` per language.
  - The existentials NLG prefers add "such that" to an NP already built, a list of kinds — agree with the last kind.

Task 23: A time explosion

- To agree with the last kind, `[ArgKind]` kept, besides its NPs, the NPs of all but the last kind and the last kind's CN, determiners and flags.
- The grammar, compiled in 1:49 before, did not finish: `CategoriesEng` at 2.7 GB after 3.5 minutes, `MathExtensionsFre` at 12 GB, swapping. A leaner record still took 9.4 minutes for French, instead of 25 s.
- Claude blamed a determiner chosen by a case, then the coordinations built at each use: both were partly right, neither was the cause.
- **My** `gf -v` showed it: `ConsArgKind` had 50,096,498,540,544 concrete functions, against 3,538,944 before; and English had got worse.

Why: GF compiles a function into one PMCFG rule for each combination of the parameters of its arguments. Every NP in a record brings its inherent agreement, every Bool doubles; and `ConsArgKind` takes an `[ArgKind]`, so the list multiplies its own combinations. A known issue of GF, but I had not expected it to hit us here.

Lesson: keep the parameters of a recursive list category few; count rules with `-v` before and after; compile one language first.

Task 23: Agreement of “such that” — outcome

`65fa1bd`, `789052b` (22 Oct)

“Let us leave out the changes in [ArgKind] for the moment”

- `suchThatCN` in each language, also in `next/`: an agreeing AP (Eng, Fre, Swe, Cze, Pol), “derart, dass” after the noun (Ger), “sellainen” before the noun and “, että” after it (Fin).
- Used where “such that” follows a CN: “un nombre naturel *n* tel que”, “přirozené číslo *n* takové, že”, “liczba naturalna *n* taka, że”.
- German `such_that_Subj`: “derart, dass”, not “sodass” (“so that”).
- Two kinds keep the conjunction: with `suchThatCN`, a variable added later came after the clause — “a reflection g such that g is left x_0 such that . . . ”, which the regression tests showed.

Remaining problems (`todo/open-issues.md`)

- The existentials that NLG prefers still say “takový, že”, “tel que” for any gender and number; “il existent”.
- English “, such that” and “such that” now both occur: 72 readings became 144.

Observations and the future

What we observed

Worked well

- Bulk lexical work with parallel subagents (2000 terms in minutes).
- Tireless crash-driven iteration (Czech RGL).
- Self-audits *when asked*: the Czech paradigm check.
- Grounding claims in measurements and diffs.
- Code in the style of the surrounding code.

Needed a human

- Design decisions: unique macro names, symbolic preference, "just time out and move on".
- `linking ...  OK` $\neq$ correct.
- Careless moments: files swept into a commit, an installed binary overwritten mid-comparison.
- Native-speaker judgement: none of the new words is reviewed.

The division of labour: I set direction and judge; Claude explores, builds, measures.

What the later tasks added

Worked well

- Tests paid off at once: a type error found within minutes of the suite.
- A second book found bugs the first one hid (`matrixProduct`, 50 times).
- Measuring: profiling first, "same output" checked on the whole book.
- Changes that did not pay off were reverted, not kept.
- A repaired check found the next bug at once ("0r").
- A fresh agent as the test of written instructions.

Needed a human

- Design: "symbolic constants are between dollars".
- The right question: *did you try -synonyms and -symbolics?*
- The idea: *don't unlex when ranking*.
- A push taken from "ready to push I guess".
- A workaround (rewording a comment) that hid a bug for two weeks.

What the latest tasks added

Worked well

- A test in all seven languages before going on: it found the word order of German and Finnish.
- Asking the design questions, with a recommendation.
- A new project, with the Informath fixes it needed kept small and tested.

Needed a human

- The diagnosis of the time explosion: `gf -v`, and knowing GF.
- Knowing when to stop: leave the hard case for later, keep what works.
- Load the grammar once: an index, not a call per statement.
- "Don't add German yet."

Speculation: what would be good to do next

- **Review**: native checking of the Finnish, Czech, Polish and French words; agreement of "such that" after a CN (Task 23); next, after a list of kinds, without a parameter explosion.
- **Tests**: a regression suite exists (Task 14); next, CI.
- **Scale**: big statements get English (Task 16); next, linearization, now the slowest part; and a lexer and unlexer that agree.
- **Proofs**: lemmas, conclusions and the statement's variables right (Task 21); next, English theorem headings, and rules that read as tautologies.
- **Coverage**: run GodementSyntax on all 10 413 sentences and measure how much of the book parses; port it to French, German, Swedish.
- **Autoformalization**: parse Godement's own text into Dedukti and type-check it against the hand-made formalization.
- **Hybrid**: an LLM proposes symbol-table entries and grammar rules, the grammar and the type checker verify them.

Thank you!

github.com/GrammaticalFramework/informath